

# K-Means Algorithm

Complete Study Notes — Concepts, Algorithm, Math, Worked Example & Model Selection

## WHAT'S INSIDE

- Introduction & intuition behind K-Means clustering
- The objective function (within-cluster sum of squares)
- Step-by-step algorithm (Lloyd's algorithm)
- Step-by-step worked numerical example
- Choosing K — Elbow method & Silhouette score
- K-Means++ initialization
- Advantages, limitations & real-world applications
- Comparison with Hierarchical Clustering
- Practice questions with answers

# 1. Introduction to K-Means Clustering

## DEFINITION

K-Means is an unsupervised machine learning algorithm that partitions a dataset into  $k$  distinct, non-overlapping clusters, where each data point belongs to the cluster with the nearest mean (centroid). It is a **centroid-based, flat (non-hierarchical) clustering** method.

K-Means is one of the simplest and most widely used clustering algorithms. Given a set of data points and a chosen number of clusters  $k$ , the algorithm iteratively assigns each point to the closest cluster centroid and then recalculates the centroids, repeating until the assignments stabilize.

## 1.1 Key Idea

The algorithm represents each cluster by a single point — its **centroid** (the mean of all points currently in that cluster). A data point is assigned to whichever cluster has the nearest centroid. The "best" clustering is the one that minimizes the total squared distance between points and their assigned centroids.

## KEY IDEA

Unlike hierarchical clustering, K-Means requires the number of clusters  $k$  to be specified *before* running the algorithm. It produces one flat partition of the data rather than a nested hierarchy.

## 1.2 The Objective Function

K-Means tries to minimize the **Within-Cluster Sum of Squares (WCSS)**, also called **inertia** — the sum of squared distances between each point and the centroid of its assigned cluster.

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

where  $C_i$  is cluster  $i$ , and  $\mu_i$  is the centroid (mean) of cluster  $i$ .

Lower WCSS means points are closer to their centroids — i.e., tighter, more compact clusters. K-Means tries to find the cluster assignment and centroid positions that make this value as small as possible.

## 2. The K-Means Algorithm (Lloyd's Algorithm)

### 2.1 Step-by-Step Procedure

- 1 Choose the number of clusters `k`.
- 2 Randomly initialize `k` centroids (often by picking `k` random data points, or using K-Means++).
- 3 **Assignment step:** assign every data point to the nearest centroid (usually using Euclidean distance), forming `k` clusters.
- 4 **Update step:** recompute each centroid as the mean of all points currently assigned to its cluster.
- 5 Repeat steps 3 and 4 until centroids no longer change significantly (convergence) or a maximum number of iterations is reached.

EXAM TIP

K-Means is also called **Lloyd's Algorithm**. It always converges (WCSS never increases between iterations), but it may converge to a **local minimum** rather than the global optimum, depending on initialization.

### 2.2 Distance Metric

K-Means conventionally uses **Euclidean distance** to measure closeness between a point and a centroid:

$$d(x, \mu) = \sqrt{\sum (x_j - \mu_j)^2}$$

Because the update step uses the arithmetic mean, K-Means is mathematically tied to Euclidean distance — using a different distance measure (e.g. Manhattan) would require a variant such as K-Medians or K-Medoids.

### 2.3 Centroid Initialization

| Method                | Description                                                                                                                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Random (Forgy method) | Pick K data points at random from the dataset as the initial centroids.                                                                                                                                            |
| Random Partition      | Randomly assign each point to one of K clusters, then compute initial centroids as the means.                                                                                                                      |
| K-Means++             | Choose centroids one at a time, each subsequent centroid chosen with probability proportional to its squared distance from the nearest existing centroid — spreads out initial centroids and improves convergence. |

WHY IT MATTERS

Poor initialization can cause K-Means to converge to a bad local minimum. In practice, the algorithm is often run multiple times with different random initializations (e.g. `n_init=10` in scikit-learn), and the run with the lowest WCSS is kept.

### 3. Worked Numerical Example

Consider 6 one-dimensional points: 2, 4, 10, 12, 3, 20. We will cluster them into  $K = 2$  clusters using K-Means.

#### 3.1 Step 1: Initialize Centroids

Suppose we randomly pick the initial centroids as  $\mu_1 = 2$  and  $\mu_2 = 10$ .

#### 3.2 Step 2: Iteration 1 — Assignment

| Point | Distance to $\mu_1$ (2) | Distance to $\mu_2$ (10) | Assigned Cluster |
|-------|-------------------------|--------------------------|------------------|
| 2     | 0                       | 8                        | C1               |
| 4     | 2                       | 6                        | C1               |
| 10    | 8                       | 0                        | C2               |
| 12    | 10                      | 2                        | C2               |
| 3     | 1                       | 7                        | C1               |
| 20    | 18                      | 10                       | C2               |

#### 3.3 Step 3: Iteration 1 — Update Centroids

$C1 = \{2, 4, 3\} \rightarrow \text{new } \mu_1 = (2+4+3)/3 = 3$

$C2 = \{10, 12, 20\} \rightarrow \text{new } \mu_2 = (10+12+20)/3 = 14$

#### 3.4 Step 4: Iteration 2 — Re-Assignment

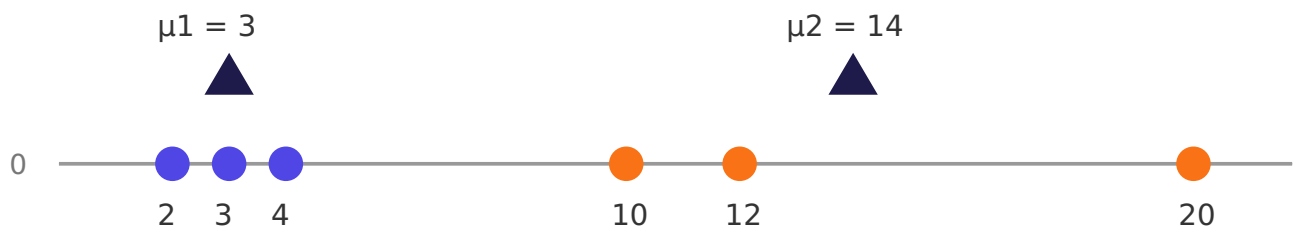
| Point | Distance to $\mu_1$ (3) | Distance to $\mu_2$ (14) | Assigned Cluster |
|-------|-------------------------|--------------------------|------------------|
| 2     | 1                       | 12                       | C1               |
| 4     | 1                       | 10                       | C1               |
| 10    | 7                       | 4                        | C2               |
| 12    | 9                       | 2                        | C2               |
| 3     | 0                       | 11                       | C1               |
| 20    | 17                      | 6                        | C2               |

The cluster assignments are unchanged from iteration 1 ( $C1 = \{2,4,3\}$ ,  $C2 = \{10,12,20\}$ ), so the centroids will not move further. **The algorithm has converged.**

EXAM TIP

Convergence is reached when either: (a) cluster assignments stop changing between iterations, or (b) centroid positions stop changing (within some tolerance). Always state clearly in your answer which condition signals convergence.

#### 3.5 Final Result



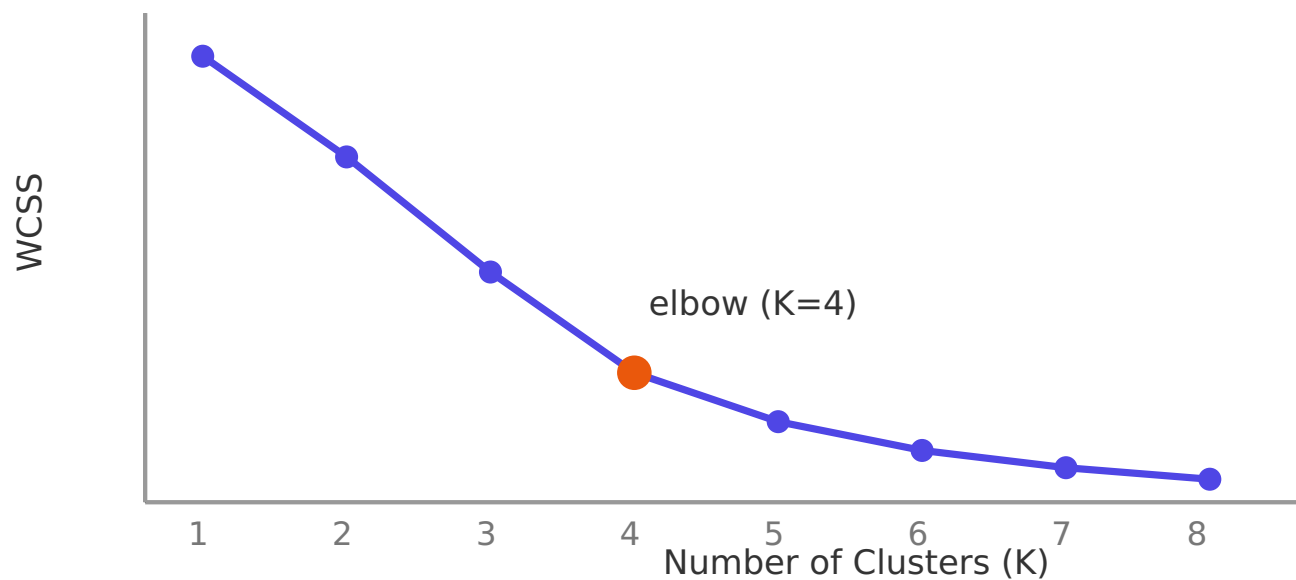
*Final clusters:  $C_1 = \{2, 3, 4\}$  with centroid 3 (blue),  $C_2 = \{10, 12, 20\}$  with centroid 14 (orange).*

## 4. Choosing the Optimal Number of Clusters (K)

Since K must be chosen in advance, selecting a good value is critical. Two of the most common techniques are the Elbow Method and the Silhouette Score.

### 4.1 The Elbow Method

Run K-Means for a range of K values (e.g. 1 to 10), recording the WCSS for each. Plot K against WCSS. As K increases, WCSS always decreases (more clusters = points closer to their own centroid), but the rate of decrease slows down after the "true" number of clusters. The point where the curve bends sharply — like an elbow — suggests a good value of K.



The "elbow" marks the K beyond which WCSS reduction flattens out significantly.

### 4.2 Silhouette Score

The silhouette score measures how well each point fits within its assigned cluster compared to other clusters. For a point  $x$ :

$$s(x) = (b(x) - a(x)) / \max(a(x), b(x))$$

where  $a(x)$  = average distance from  $x$  to other points in its own cluster, and  $b(x)$  = average distance from  $x$  to points in the nearest neighboring cluster.

| Silhouette Score Range | Interpretation                                                              |
|------------------------|-----------------------------------------------------------------------------|
| Close to +1            | Point is well-matched to its own cluster, far from others — good clustering |
| Close to 0             | Point lies on or very near the boundary between two clusters                |
| Close to -1            | Point is likely assigned to the wrong cluster                               |

The average silhouette score across all points is computed for each candidate K, and the K with the highest average score is generally preferred.

EXAM TIP

The Elbow Method is visual and somewhat subjective, while the Silhouette Score gives a single quantitative number for each K, making it easier to compare candidate values objectively.

## 5. K-Means++ Initialization

Standard K-Means with random initialization can sometimes pick poor starting centroids, leading to slow convergence or a bad local minimum. **K-Means++** is a smarter initialization strategy designed to fix this.

## 5.1 K-Means++ Procedure

- 1 Choose the first centroid uniformly at random from the data points.
- 2 For each remaining data point, compute its squared distance to the nearest already-chosen centroid.
- 3 Choose the next centroid randomly, with probability proportional to this squared distance (points farther from existing centroids are more likely to be chosen).
- 4 Repeat steps 2-3 until all K centroids are chosen.
- 5 Proceed with standard K-Means assignment/update steps as usual.

### WHY IT HELPS

By spreading initial centroids further apart, K-Means++ reduces the chance of two centroids starting in the same true cluster, leading to faster convergence and more reliable (better quality) clustering results.

## 6. Advantages and Limitations

| ADVANTAGES                                                                                                                                                                                                                                                                                                                                                             | LIMITATIONS                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Simple to understand and implement</li><li>Computationally efficient — scales well to large datasets, <math>O(n \cdot K \cdot i \cdot d)</math></li><li>Works well when clusters are roughly spherical and similarly sized</li><li>Guaranteed to converge (WCSS decreases monotonically)</li><li>Easily parallelizable</li></ul> | <ul style="list-style-type: none"><li>Must specify K in advance</li><li>Sensitive to initial centroid placement — can converge to a local minimum</li><li>Assumes clusters are convex/spherical and similar in size — struggles with irregular shapes</li><li>Sensitive to outliers (they can pull centroids away from the true cluster center)</li><li>Sensitive to feature scaling — features with larger ranges dominate distance calculations</li></ul> |

Where  $n$  = number of points,  $K$  = number of clusters,  $i$  = number of iterations, and  $d$  = number of dimensions/features.

**EXAM TIP**

Always mention that features should be **standardized/normalized** before applying K-Means, since the algorithm relies on Euclidean distance, which is scale-sensitive.

## 7. K-Means vs Hierarchical Clustering

| Aspect                  | K-Means                                                | Hierarchical Clustering                                             |
|-------------------------|--------------------------------------------------------|---------------------------------------------------------------------|
| Number of clusters (K)  | Must be specified in advance                           | Not required in advance; chosen after viewing dendrogram            |
| Output                  | A single flat partition                                | A complete hierarchy / dendrogram                                   |
| Reproducibility         | Depends on random centroid initialization              | Deterministic (same result every run)                               |
| Scalability             | Scales well to large datasets — $O(n \cdot K \cdot i)$ | Poor for very large datasets — $O(n^2)$ or worse                    |
| Cluster shape           | Assumes roughly spherical, equally-sized clusters      | Can capture nested, non-spherical structures (depending on linkage) |
| Sensitivity to outliers | Sensitive — outliers distort centroids                 | Single linkage sensitive; Ward's/complete linkage less so           |

## 8. Real-World Applications

- Customer Segmentation:** grouping customers by purchasing behavior for targeted marketing.
- Image Compression:** reducing the number of distinct colors in an image by clustering pixel colors.
- Document Clustering:** grouping similar documents or articles by topic.
- Anomaly Detection:** points far from any centroid can be flagged as potential outliers/anomalies.
- Geographic/Spatial Analysis:** identifying regional clusters, e.g. for delivery hub placement.
- Recommendation Systems:** grouping users with similar preferences for collaborative filtering.

## 9. Summary & Quick Revision

### Key Takeaways

- K-Means partitions data into  $K$  flat (non-nested) clusters based on nearest centroid.
- It minimizes the **Within-Cluster Sum of Squares (WCSS)**, also called inertia.
- The algorithm alternates between an **assignment step** and an **update step** until convergence.
- $K$  must be chosen in advance — common techniques: **Elbow Method** and **Silhouette Score**.
- **K-Means++** improves initialization by spreading centroids apart, reducing the risk of poor local minima.
- K-Means is fast and scalable ( $O(n \cdot K \cdot i)$ ) but assumes spherical, similarly-sized clusters and is sensitive to outliers and feature scaling.
- Always converges, but possibly to a **local** minimum — multiple random initializations are recommended.

### 9.1 Practice Questions

#### Q1. What does K-Means minimize?

A1. The Within-Cluster Sum of Squares (WCSS) — the total squared distance between each point and its assigned cluster centroid.

#### Q2. Why might K-Means give different results on different runs with the same $K$ ?

A2. Because centroids are typically initialized randomly, and different starting positions can lead the algorithm to converge to different local minima.

#### Q3. How does K-Means++ differ from standard random initialization?

A3. K-Means++ selects centroids one at a time, choosing each new centroid with probability proportional to its squared distance from the nearest already-chosen centroid, spreading the initial centroids out instead of choosing them purely at random.

#### Q4. What does a silhouette score close to 0 indicate about a point?

A4. The point lies near the boundary between two clusters — it is roughly equally close to its own cluster and a neighboring cluster.

#### Q5. Why is feature scaling important before running K-Means?

A5. Because K-Means relies on Euclidean distance; features with larger numeric ranges would dominate the distance calculation and bias the clustering unless all features are standardized/normalized first.

#### Q6. What are the two conditions that signal K-Means has converged?

A6. Either the cluster assignments no longer change between iterations, or the centroid positions stop changing (within a small tolerance) between iterations.